

Greenhouse Management Report

This is a very complex topic that is associated with all-around considerations in plant health, inventory control, greenhouse organization, and staff coordination. There is more than one key factor involved: managing the life cycle of plants, strategies for watering, and customer service/interaction. A greenhouse should thrive on maintaining healthy plants, giving good customer experiences, and achieving operational efficiency. A greenhouse has to evolve over a period of time: expanding inventories and making necessary modifications to its systems to meet seasonal changes and demands of the customers.

Basic requirements of a greenhouse include proper plant care systems for maintaining plant health throughout their lifecycle. Storage facilities are needed to organize plants by type, size, and care requirements. Staff management systems are essential to coordinate daily tasks such as watering, pruning, fertilizing, and assisting customers. All these components are key to the greenhouse's success. The greenhouse economy depends on maintaining healthy inventory to minimize loss and providing value added services like decorative pots, gift wrapping, and custom arrangements.

Using the Composite Design Pattern, a greenhouse can be divided into smaller pieces or sections. This will help with management regarding plant types that need very specific environmental conditions. A greenhouse is managed by staff who are responsible for plant care and customer service. The management system must support both routine maintenance tasks and dynamic responses to the changing needs of plants as they progress through different lifecycle stages.

It will be important that this greenhouse integrates both legacy systems-like traditional irrigation-with modern automation. The layout of the facility should efficiently organize plants for easy access, maintain environmental zones appropriately, and streamline staff workflows through automation of tasks and clear scheduling. Inventory management will have to track each individual plant and its health status, the care history of the plants, and their location in the facility.

The life cycle of a greenhouse can be described as moving plants between various stages: seedling, growing, mature, flowering, and dormant. Each of these stages may require a different kind of care. Some plants must have much more water than others. Sometimes the plants are sold individually, but often they are combined in more involved presentations that include a lot of decorations. The system should be able to provide support not only for simple retail sales but also for custom arrangement orders.

Knowing all of this, we designed our system around these key components: A comprehensive plant hierarchy with multiple specialized types (roses, cacti, succulents, lavender, baobab trees). A state management system to handle plant lifecycle transitions. A strategy system for flexible watering approaches. Staff task automation through command-based scheduling. Hierarchical facility organization with composite greenhouse sections. Inventory management with efficient

traversal mechanisms. Plant product decoration for retail sales. Complex arrangement building for custom orders. Integration with legacy irrigation systems. Plant propagation through prototyping.

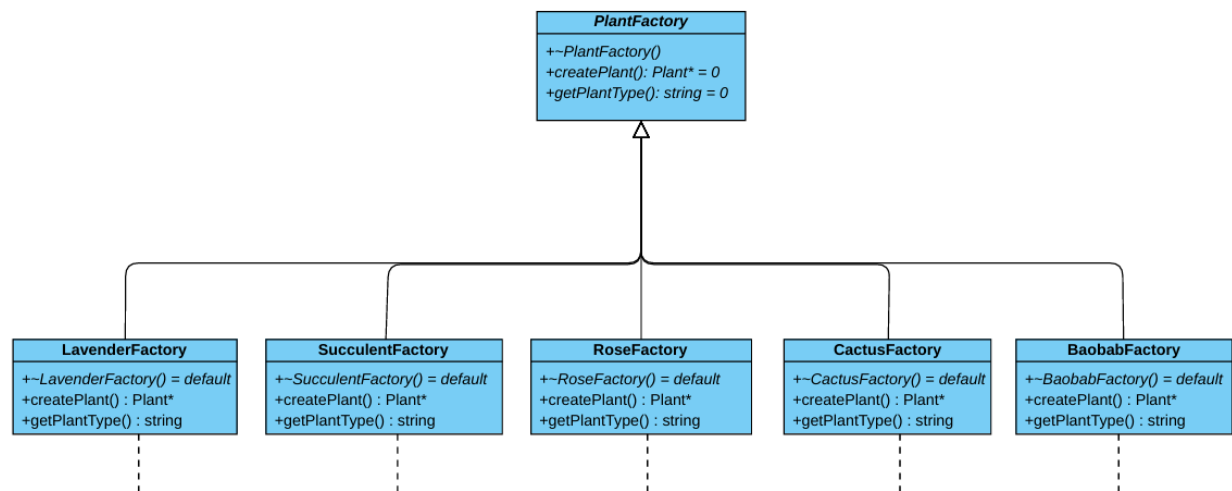
Design Pattern Application Report

Overview

We modeled our greenhouse around a **Plant** base class. This class serves as the foundation for all plant types in the system and integrates multiple design patterns to provide comprehensive functionality. Plant was then extended to create specialized types: **Rose**, **Cactus**, **Succulent**, **Lavender**, and **Baobab**. Each plant type has unique care requirements, growth characteristics, and customer appeal.

To instantiate these plants, we used the **Factory Method** pattern. We chose the Factory Method pattern because it lets a class defer instantiation to subclasses. This means the client code (such as inventory managers or sales staff) only needs to know about the abstract **PlantFactory** class without concerning itself with the specific details of creating roses or cacti or succulents. This provides excellent expandability as adding a new plant type only requires creating a new factory subclass without modifying existing code.

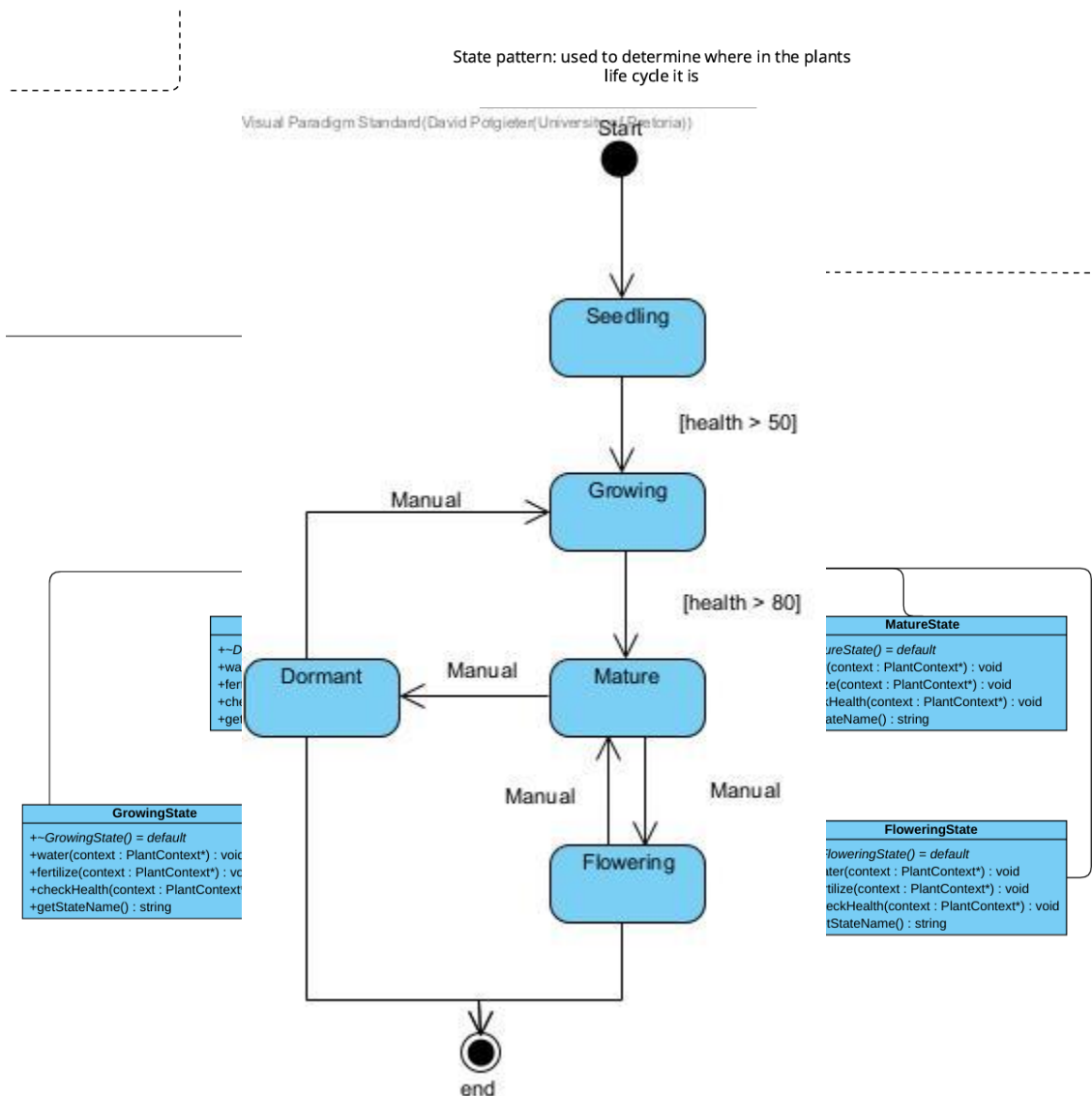
Factory method for easy creation of plants



Plant Lifecycle Management: State Pattern

Plants progress through distinct lifecycle stages, each requiring different care approaches. We implemented the **State** design pattern to allow plants to alter their behavior based on their current lifecycle state. The five states are:

The **PlantContext** class maintains a reference to the current state and delegates all care operations (water, fertilize, checkHealth) to the current state object. State transitions occur either automatically (health-based thresholds) or manually (seasonal changes). This allows us to easily expand the system with additional states without modifying the Plant class itself.



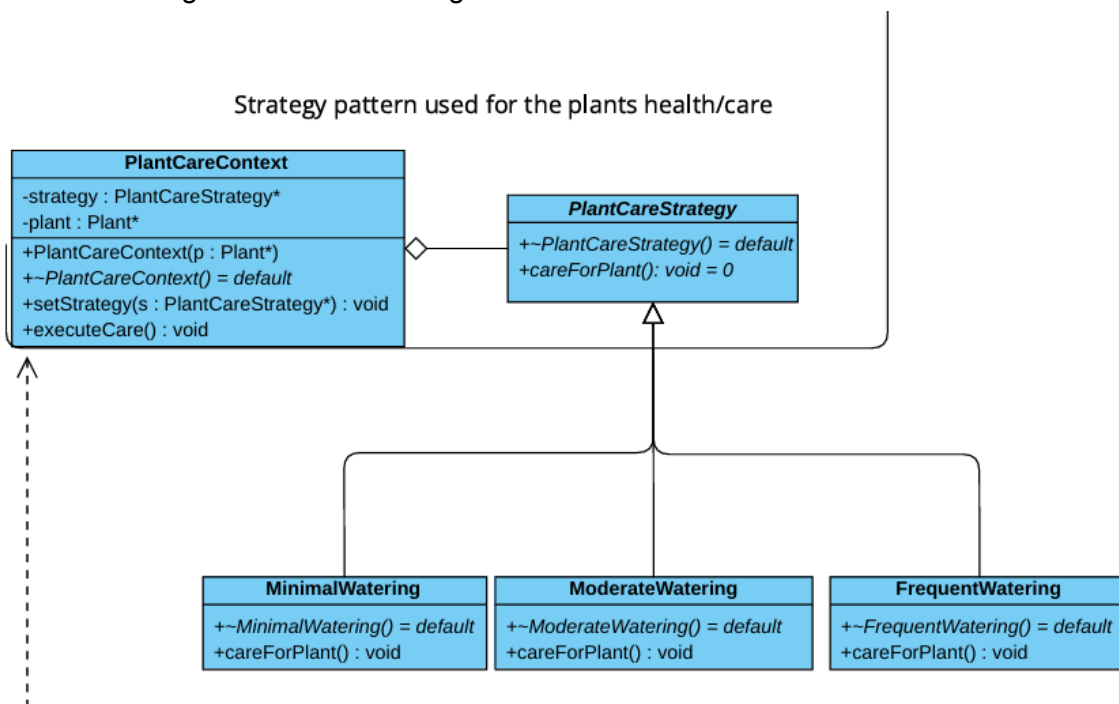
Flexible Care Approaches: Strategy Pattern

Different plant types require vastly different watering schedules. Cacti need minimal water while tropical plants require frequent watering. We implemented the **Strategy** design pattern to handle these varying care approaches independently from the plant objects themselves.

The **PlantCareStrategy** abstract class defines the interface with concrete implementations:

- FrequentWatering - Daily watering schedule for water-loving plants
- ModerateWatering - Regular schedule for typical plants
- MinimalWatering - Sparse watering for drought-tolerant plants like succulents

Each Plant object holds a reference to a PlantCareStrategy, which can be changed at runtime. This allows the watering algorithm to vary independently from the plant doing the consuming. For example, a succulent might start with MinimalWatering but could be switched to ModerateWatering if moved to a drier greenhouse section.



Standardized Care Routines: Template Method Pattern

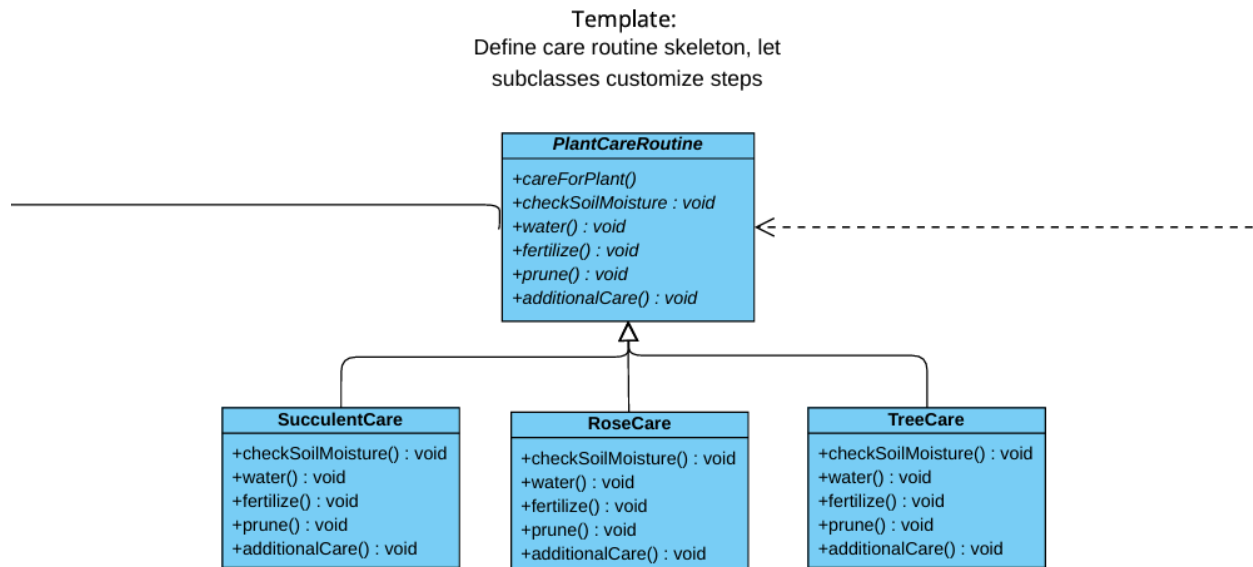
While care strategies define watering frequency, the overall care algorithm follows a consistent structure across all plant types. We used the **Template Method** pattern to define the skeleton of the plant care algorithm while allowing individual steps to be customized per plant type.

The abstract PlantCareRoutine class defines the template method careForPlant() which executes the following sequence:

1. checkSoilMoisture() - Verify current moisture levels (primitive operation)
2. water() - Apply appropriate water amount (primitive operation)
3. fertilize() - Apply plant-specific fertilizer (primitive operation)
4. prune() - Perform pruning if needed (primitive operation)

5. additionalCare() - Optional hook for special care requirements

Concrete implementations override the primitive operations to provide plant-specific care while maintaining the overall algorithm structure.

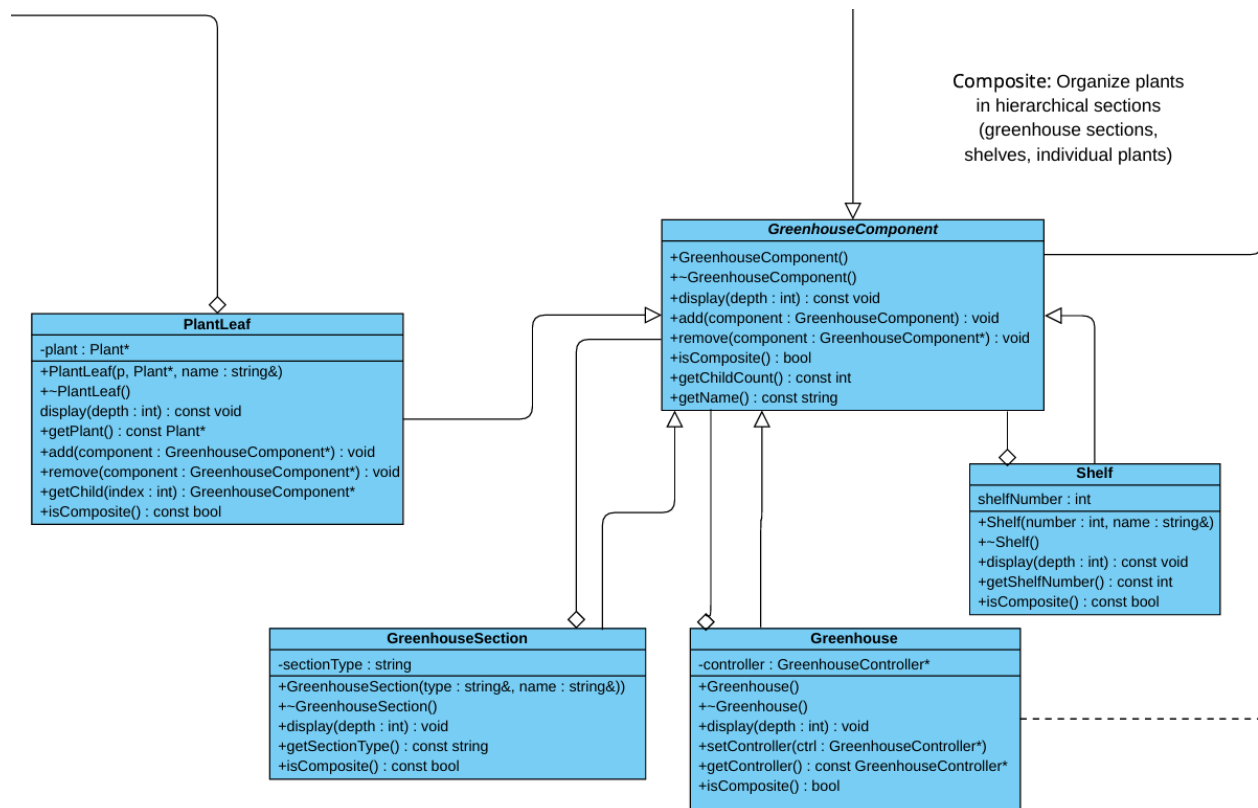


Hierarchical Facility Organization: Composite Pattern

Our greenhouse facility has a hierarchical structure: the entire Greenhouse contains multiple GreenhouseSections each section contains multiple Shelves, and each shelf contains individual PlantLeaf objects.

We use the **Composite** design pattern to treat individual plants and groups of plants uniformly. The abstract GreenhouseComponent class defines operations like `display()`, `add()`, `remove()`, and `getChild()`. Composite classes (Greenhouse, GreenhouseSection, Shelf) can contain child components, while PlantLeaf represents leaf nodes.

This pattern allows us to perform operations on the entire facility, a specific section, or an individual plant using the same interface. This makes facility-wide operations like "water all plants" or "count total plants" simple recursive calls.

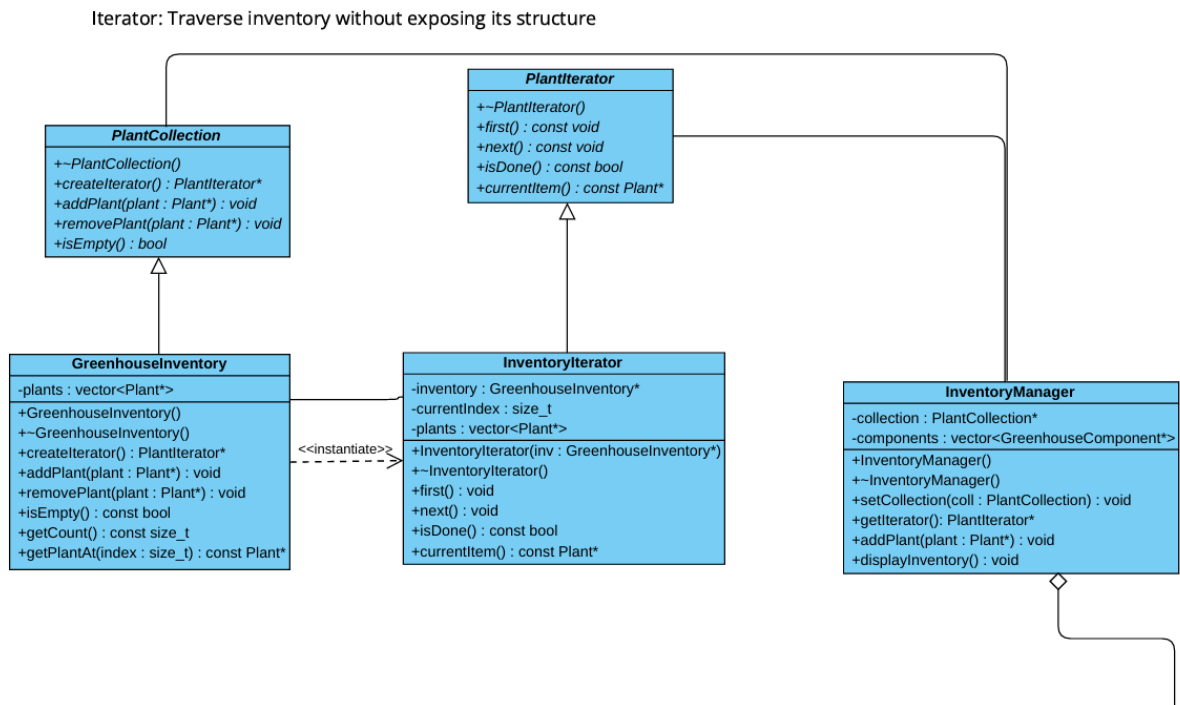


Inventory Traversal: Iterator Pattern

The greenhouse inventory stores plants in a collection (vector), but we don't want to expose the underlying data structure to client code. We implemented the **Iterator** design pattern to traverse the plant collection without exposing its internal representation.

An important design decision was to create snapshot iterators, when an iterator is created, it captures a copy of the current plant pointers. This prevents issues if the inventory is modified during iteration. The **InventoryManager** class serves as a facade providing high-level inventory operations while internally using iterators for traversal.

This pattern allows us to change the internal storage mechanism (from vector to list to hash map) without affecting client code that uses iterators. It also enables multiple simultaneous iterations over the same collection.

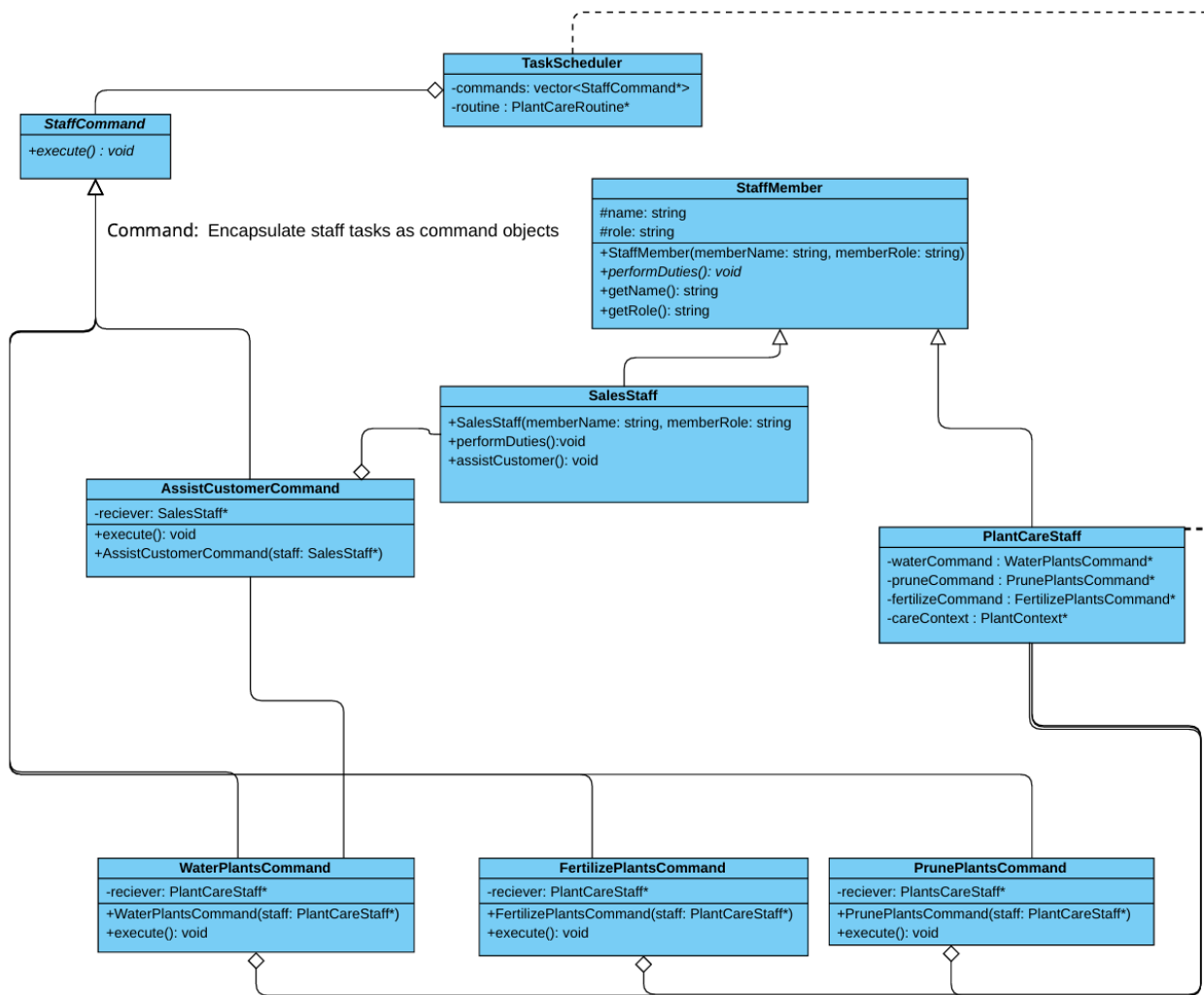


Staff Task Automation: Command Pattern

Greenhouse staff perform various tasks throughout the day: watering plants, pruning, fertilizing, and assisting customers. We use the **Command** design pattern to encapsulate these requests as objects, enabling task queuing, scheduling, logging, and potential undo functionality.

Each command holds a reference to a receiver object. The TaskScheduler class acts as the invoker, maintaining a queue of commands and executing them sequentially. It also maintains a command history for logging purposes.

This pattern decouples task requesters from task performers (staff members). It enables batch command execution, command prioritization, and maintains a record of all completed tasks.



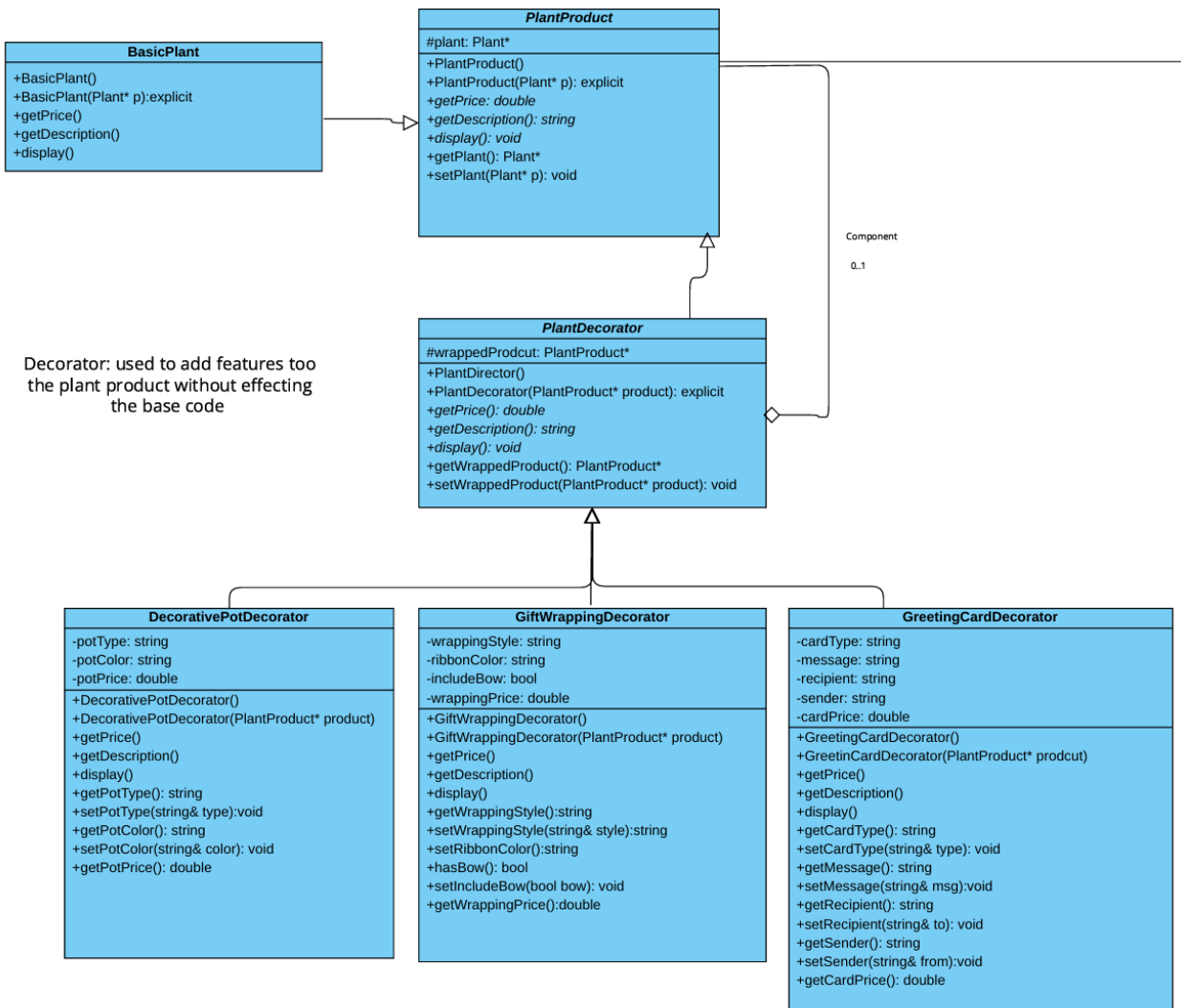
Dynamic Product Features: Decorator Pattern

When selling plants, customers often want additional features: decorative pots, gift wrapping, greeting cards. Rather than creating subclasses for every possible combination we use the **Decorator** pattern to dynamically add responsibilities to plant products.

Concrete decorators include:

- DecorativePotDecorator - Adds pot cost (R50) and description
- GiftWrappingDecorator - Adds wrapping cost (R25) and description
- GreetingCardDecorator - Adds card cost (R15) and description

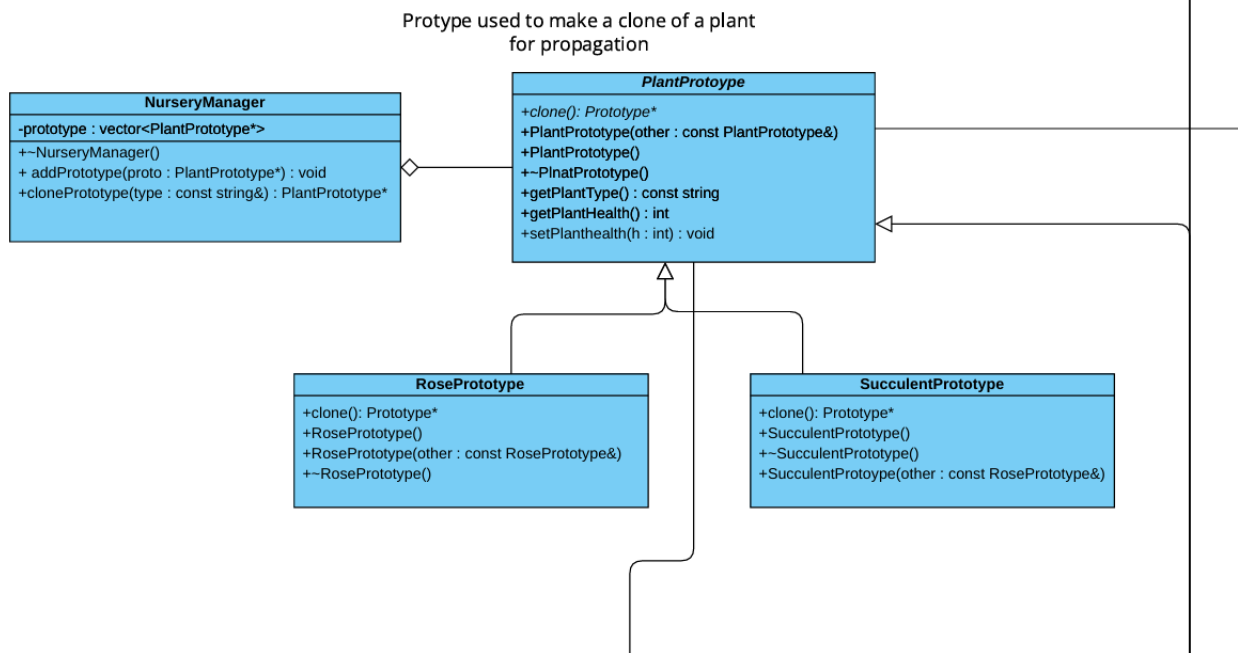
Each decorator adds its cost and description to the wrapped product.



Plant Propagation: Prototype Pattern

Greenhouse operations often involve propagating successful plant specimens cloning healthy plants to create new inventory. We implemented the **Prototype** pattern to support plant cloning without depending on concrete plant classes.

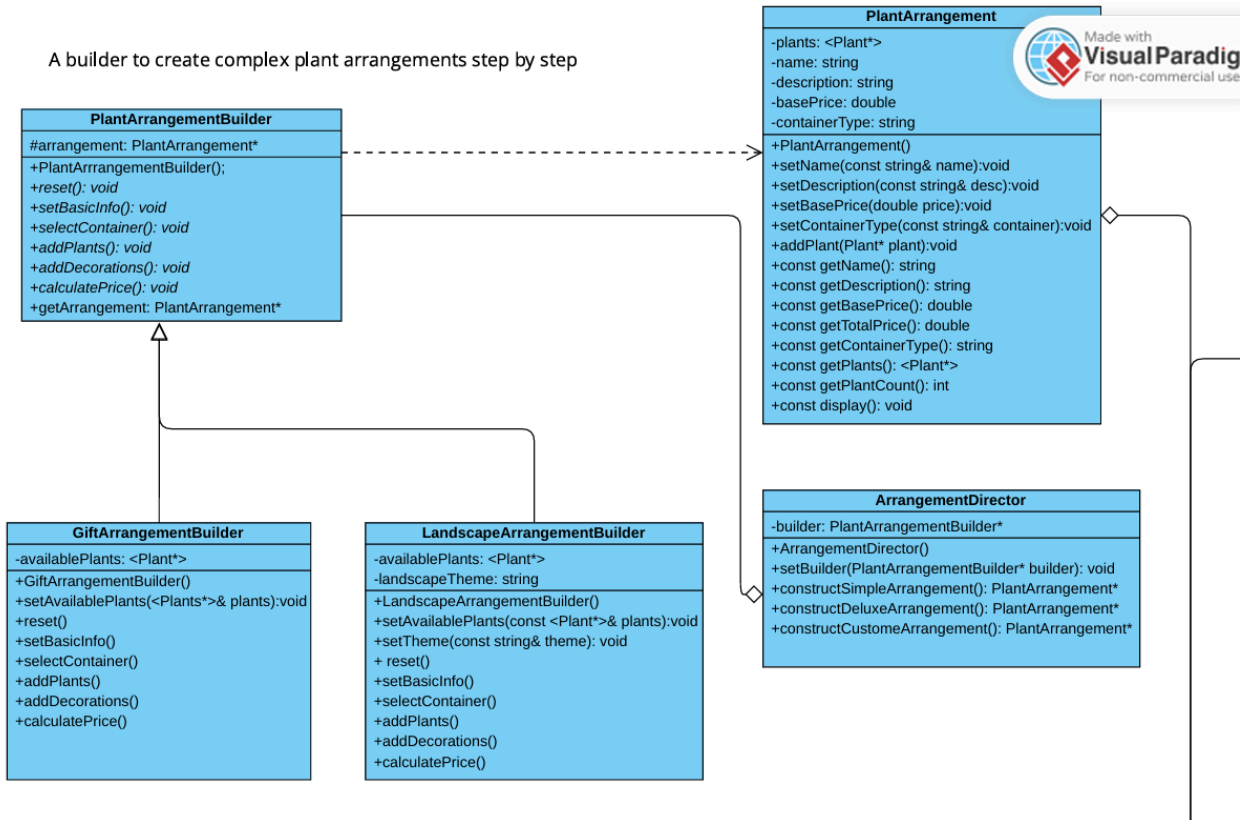
This pattern enables efficient plant propagation. Instead of creating plants from scratch, we can clone existing healthy specimens, preserving their optimal characteristics. The Plant base class itself includes the abstract clone() method, making all plant types cloneable.



Complex Product Creation: Builder Pattern

Creating complex plant arrangements involves multiple steps: selecting container type, choosing compatible plants, adding decorative elements, calculating total price, etc. We use the **Builder** pattern to construct these complex products step-by-step.

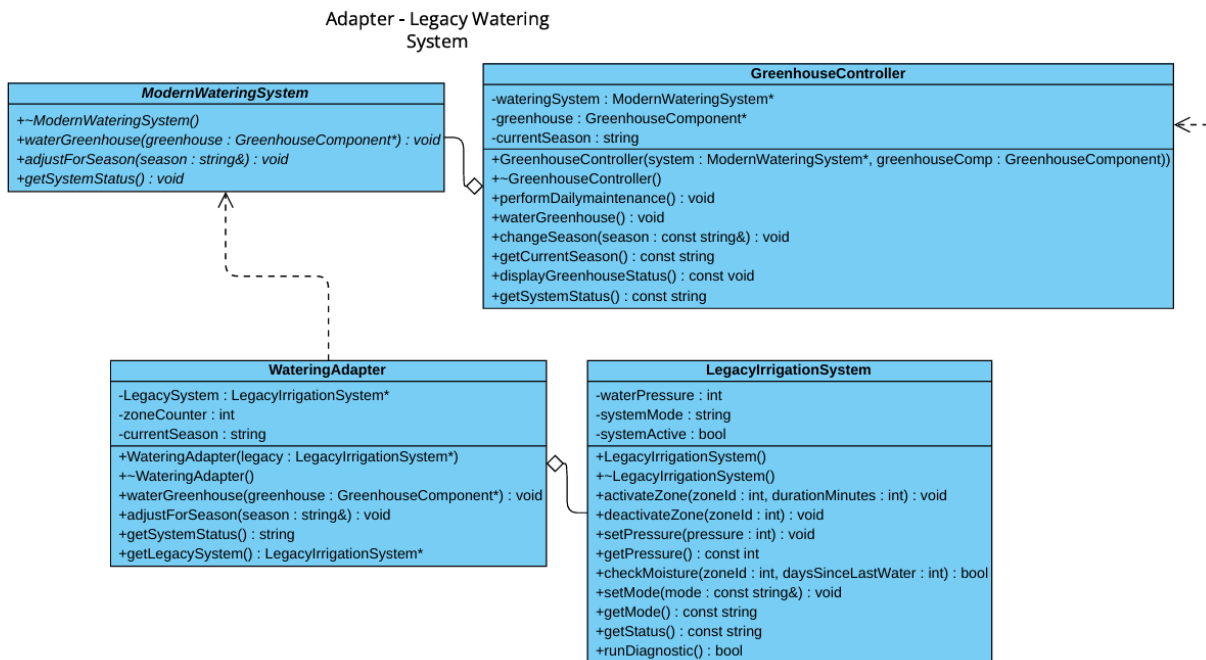
This pattern separates the construction algorithm from the product representation, allowing the same construction process to create different arrangements.



Legacy System Integration: Adapter Pattern

Our greenhouse incorporates a legacy irrigation system that predates our modern software architecture. Rather than replacing the expensive physical equipment, we use the **Adapter** pattern to make the legacy system compatible with our modern interface.

The LegacyIrrigationSystem class represents the old system with incompatible methods like setZoneWater(zoneNumber, amount) and setSeason(season). The WateringAdapter class implements the ModernWateringSystem interface while internally translating calls to the legacy system's methods.



Design Strengths

Our greenhouse management system exhibits several key architectural strengths:

1. Extensibility

Adding new plant types, care routines, staff roles, or decorations requires minimal changes to existing code. Each pattern provides extension points through abstract classes and interfaces.

3. Flexibility

Strategy and Decorator patterns enable runtime customization. Plants can change watering approaches mid-lifecycle, and products can be decorated in any combination.

4. Reusability

Pattern implementations are independent and can be reused in other contexts. The Command pattern task scheduler could easily be adapted for other scheduling domains.

6. Scalability

Composite and Iterator patterns handle collections of any size efficiently. The system scales from small boutique greenhouses to large commercial operations.

